Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский
политехнический университет»**

Электротехнический факультет
Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: 09.03.01 – «Информатика, вычислительная техника»

Дискретная математика и математическая логика
Лабораторная работа №3

Выполнил студент гр. ИВТ-24-1б:

Оглезнев Никита Михайлович
(Фамилия, Имя, Отчество)

(подпись)

Проверил:

ст. преп. каф. ИТАС
Рустамханова Г. И.
(должность, Ф.И.О руководителя от кафедры)

(оценка) (подпись)

(дата)

Пермь 2026

Содержание

Содержание 2
Цель 3
Задачи 3
Теоретический блок 4
Реализация 5
Заключение 6
Приложение А 7
Приложение Б 8

Цель

Изучить алгебраический подход к поиску путей в графах и реализовать алгоритм Шимбелла для нахождения путей минимальной или максимальной длины при фиксированном количестве переходов.

Задачи

1. Реализовать механизм возведения весовой матрицы графа в степень с использованием специфических операций сложения и выбора экстремума (min/max).
2. Создать систему ввода матрицы смежности 6x6 (клавиатура, файл или встроенный шаблон) с валидацией данных.
3. Обеспечить возможность переключения режимов поиска между поиском кратчайшего (min-plus) и самого длинного (max-plus) путей.
4. Вывести итоговую матрицу путей, где каждый элемент a_{ij}соответствует экстремальному весу пути из вершины I в вершину j ровно за k шагов.

Теоретический блок

Алгоритм Шимбелла базируется на аналогии между обычным умножением матриц и процессом поиска путей в графе. Если в стандартной алгебре при умножении матриц мы используем операции (+, x),то в задачах на графах для поиска путей длины k используются другие алгебраические структуры:

- Для поиска минимального пути (тропическое полукольцо): операция сложения заменяется на нахождение минимума (min), а умножение — на сложение (+).
- Для поиска максимального пути: операция сложения заменяется на нахождение максимума (max), а умножение остается сложением (+).

Математически: если A^1 – исходная матрица весов, то $A^k = A^{k-1} \otimes A^1$ содержит веса оптимальных путей, состоящих ровно из k ребер. При этом отсутствие ребра в «минимуме» интерпретируется как $+\infty$, а в «максимуме» как $-\infty$.

Реализация

Программа построена как последовательная обработка графа, начиная с подготовки весовой матрицы и выбора параметров поиска. На первом этапе функция read_matrix_from_file или блок ручного ввода обеспечивают получение данных размером 6x6.

Важным нюансом является предварительная трансформация: поскольку в исходной матрице 0 означает отсутствие пути, программа заменяет эти нули на бесконечность (положительную для min и отрицательную для max), чтобы они не искажали результаты вычислений.

Основная вычислительная логика инкапсулирована в функции shimbell_multiply, которая выполняет специфическое умножение двух матриц. Вместо стандартного вычисления скалярного произведения строк и столбцов, здесь для каждой пары вершин (i, j) ищется промежуточная вершина t, минимизирующая или максимизирующая суммарный вес перехода.

Функция matrix_power управляет процессом возведения в степень. Она принимает количество переходов k и последовательно применяет умножение Шимбелла. Если пользователь вводит k=0, программа возвращает аналог единичной матрицы, где расстояние от вершины до самой себя равно 0, а до всех остальных — бесконечность.

Завершается работа программы блоком форматированного вывода. Перед выводом бесконечные значения (означающие, что путь ровно за k шагов не существует) преобразуются обратно в 0 для удобства чтения. Таким образом, пользователь получает четкую таблицу, где наглядно представлены веса всех возможных маршрутов заданной длины.

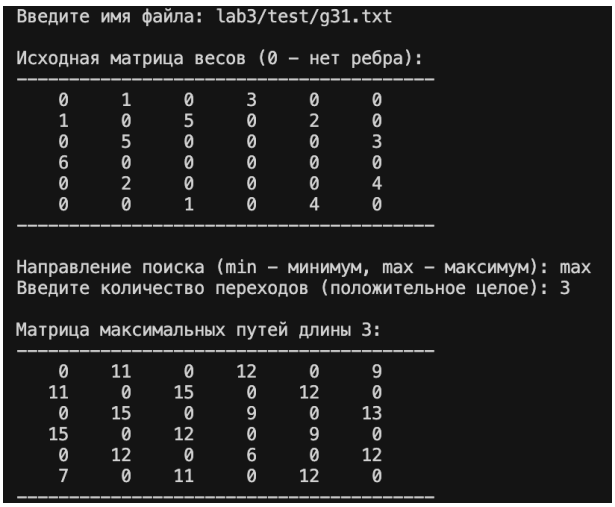
Заключение

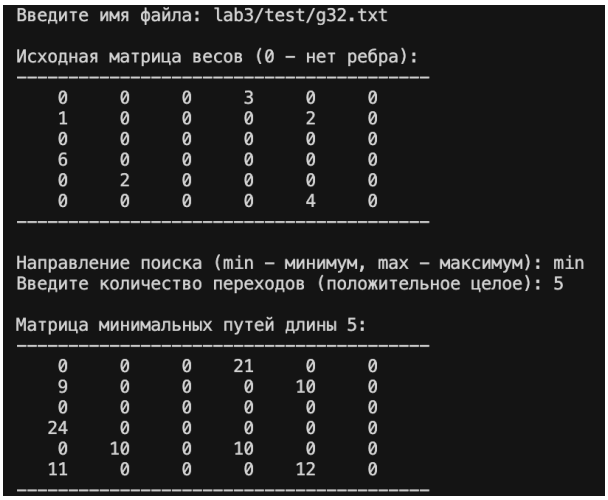
В ходе лабораторной работы был успешно реализован алгоритм Шимбелла, демонстрирующий эффективность матричного подхода к задачам на графах.

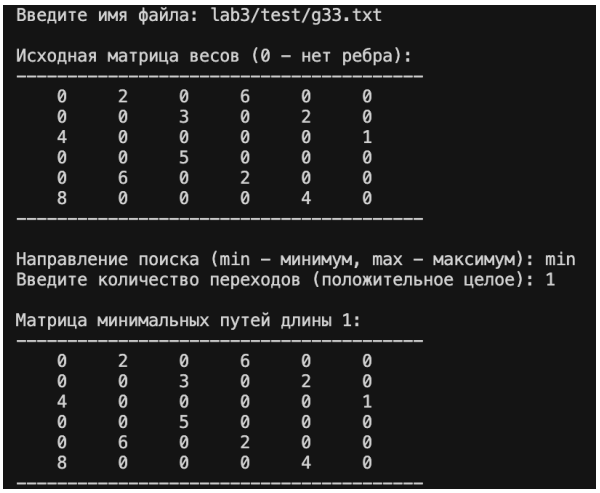
Программа позволяет находить как кратчайшие, так и максимально длинные пути с фиксированным числом переходов, что полезно в задачах логистики и планирования маршрутов с заданными ограничениями.

Приложение А

Результаты работы программы на тестовых файлах.







Приложение Б

Полный код программы.

```
"""
Алгоритм Шимбелла для поиска путей минимальной или максимальной длины
с заданным числом переходов.
Вход: весовая матрица 6x6 (0 – отсутствие пути)
Выход: матрица 6x6 с весами наилучших путей длины k.
"""

import math

def read_matrix_from_file(filename):
    """Чтение матрицы 6x6 из файла."""
    try:
        with open(filename, 'r') as f:
            matrix = []
            for line_num, line in enumerate(f, 1):
                line = line.strip()
                if not line:
                    continue
                parts = line.split()
                if len(parts) != 6:
                    raise ValueError(f"Строка {line_num}: ожидается 6 чисел, получено
{len(parts)}")
                row = [int(x) for x in parts]
                if any(x < 0 for x in row):
                    raise ValueError(f"Строка {line_num}: отрицательные числа не
допускаются")
                matrix.append(row)
            if len(matrix) != 6:
                raise ValueError(f"Должно быть 6 строк, получено {len(matrix)}")
            return matrix
    except Exception as e:
        print(f"Ошибка при чтении файла: {e}")
        return None

def print_matrix(matrix, title="Матрица"):
    """Вывод матрицы."""
    print(f"\n{title}:")
    print("-" * 40)
    for row in matrix:
        print(" ".join(f"{x:5}" if x != float('inf') and x != -float('inf') else
f"{'inf':5}" for x in row))
    print("-" * 40)

def shimbell_multiply(A, B, mode='min', inf_val=float('inf')):
    """
    Умножение матриц по операциям Шимбелла.
    A, B – матрицы (списки списков) размера n x n.
    mode: 'min' для min-plus, 'max' для max-plus.
```

```
    inf_val: значение, обозначающее отсутствие пути.
    Возвращает новую матрицу C, где C[i][j] = (по mode) над t (A[i][t] + B[t][j]).
    """
    n = len(A)
    C = [[inf_val] * n for _ in range(n)]
    if mode == 'min':
        for i in range(n):
            for j in range(n):
                best = inf_val
                for t in range(n):
                    if A[i][t] != inf_val and B[t][j] != inf_val:
                        s = A[i][t] + B[t][j]
                        if s < best:
                            best = s
                if best != inf_val:
                    C[i][j] = best
    elif mode == 'max':
        neg_inf = -float('inf')
        for i in range(n):
            for j in range(n):
                best = neg_inf
                for t in range(n):
                    if A[i][t] != neg_inf and B[t][j] != neg_inf:
                        s = A[i][t] + B[t][j]
                        if s > best:
                            best = s
                if best != neg_inf:
                    C[i][j] = best
    else:
        raise ValueError("mode должен быть 'min' или 'max'")
    return C

def matrix_power(matrix, power, mode='min'):
    """
    Возведение матрицы в степень power по операциям Шимбелла.
    matrix - исходная весовая матрица (0 означает отсутствие ребра).
    power - целое неотрицательное число (количество переходов).
    Возвращает матрицу путей длины power.
    """
    n = len(matrix)
    # Преобразуем 0 в специальное значение для отсутствия ребра
    if mode == 'min':
        inf_val = float('inf')
        # для минимума: отсутствие пути - бесконечность
        mat = [[inf_val] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                if matrix[i][j] != 0:
                    mat[i][j] = matrix[i][j]
    else:  # max
        inf_val = -float('inf')
        mat = [[inf_val] * n for _ in range(n)]
```

```
        for i in range(n):
            for j in range(n):
                if matrix[i][j] != 0:
                    mat[i][j] = matrix[i][j]

    if power == 0:
        # Единичная матрица: пути длины 0 (только из вершины в себя)
        result = [[inf_val] * n for _ in range(n)]
        for i in range(n):
            if mode == 'min':
                result[i][i] = 0   # для min-пути длина 0
            else:
                result[i][i] = 0   # для max-пути тоже 0 (путь нулевой длины)
        return result

    # Начальная степень 1
    result = mat
    for _ in range(1, power):
        result = shimbell_multiply(result, mat, mode, inf_val)
    return result

def main():
    print("=" * 60)
    print("ПОИСК ПУТЕЙ МИНИМАЛЬНОЙ/МАКСИМАЛЬНОЙ ДЛИНЫ (АЛГОРИТМ ШИМБЕЛЛА)")
    print("=" * 60)

    # Ввод матрицы
    while True:
        print("\nВыберите способ ввода матрицы 6x6:")
        print("1. Ввести с клавиатуры")
        print("2. Загрузить из файла")
        print("3. Использовать встроенный пример")
        choice = input("Ваш выбор (1/2/3): ").strip()
        if choice == '1':
            print("Введите 6 строк по 6 целых неотрицательных чисел, разделённых
пробелами:")
            matrix = []
            for i in range(6):
                while True:
                    line = input(f"Строка {i+1}: ").strip()
                    parts = line.split()
                    if len(parts) != 6:
                        print("Ошибка: должно быть 6 чисел. Попробуйте снова.")
                        continue
                    try:
                        row = [int(x) for x in parts]
                        if any(x < 0 for x in row):
                            print("Ошибка: числа должны быть неотрицательными.")
                            continue
                        matrix.append(row)
                        break
                    except ValueError:
```

```
                    print("Ошибка: введите целые числа.")
            break
        elif choice == '2':
            filename = input("Введите имя файла: ").strip()
            matrix = read_matrix_from_file(filename)
            if matrix is not None:
                break
        elif choice == '3':
            # Пример матрицы 6x6
            matrix = [
                [0, 2, 0, 1, 0, 0],
                [2, 0, 3, 0, 0, 0],
                [0, 3, 0, 0, 0, 4],
                [1, 0, 0, 0, 5, 0],
                [0, 0, 0, 5, 0, 6],
                [0, 0, 4, 0, 6, 0]
            ]
            print("Используется встроенная матрица:")
            print_matrix(matrix, "Исходная матрица")
            break
        else:
            print("Неверный выбор. Попробуйте снова.")

    # Вывод исходной матрицы
    print_matrix(matrix, "Исходная матрица весов (0 - нет ребра)")

    # Выбор направления
    while True:
        direction = input("\nНаправление поиска (min - минимум, max - максимум):
").strip().lower()
        if direction in ['min', 'max']:
            break
        else:
            print("Неверный ввод. Введите 'min' или 'max'.")

    # Ввод количества переходов
    while True:
        try:
            k = int(input("Введите количество переходов (положительное целое): "))
            if k <= 0:
                print("Число должно быть положительным.")
                continue
            break
        except ValueError:
            print("Введите целое число.")

    # Построение матрицы путей длины k
    result = matrix_power(matrix, k, direction)

    # Преобразование бесконечностей в 0 для вывода
    output_matrix = [[0] * 6 for _ in range(6)]
    for i in range(6):
```

```
        for j in range(6):
            val = result[i][j]
            if direction == 'min':
                if val != float('inf'):
                    output_matrix[i][j] = val
            else:
                if val != -float('inf'):
                    output_matrix[i][j] = val

    # Вывод результата
    print_matrix(output_matrix, f"Матрица {'минимальных' if direction == 'min' else
'максимальных'} путей длины {k}")

    print("\n" + "=" * 60)
    print("готово")
    print("=" * 60)

if __name__ == "__main__":
    main()
```